微机原理复习

1. 计算机系统概述

1.1. 内存？外存？

（1）内存：包含 RAM，ROM，CACHE。性质：易失性（ROM 不易失）。Cpu 直接访问。速度快，容量小，价格高

（2）外存：包括磁盘，光盘等，非易失性。不能被 cpu 直接访问。速度慢，容量大，价格低。

（3）拓展（第六章）：RAM：随机读写存储器

ROM：只读存储器

1.2. 冯诺伊曼结构：

（1） 计算机：运算器，存储器，控制器，输入设备，输出设备

（2） 指令和数据均以二进制表示，程序和数据放在同一存储器，指令按顺序存放，计算机以运算器为中心

（2）冯诺伊曼的计算机以运算器为中心，现代计算器以存储器为中心。

存储程序控制：编好的程序事先存入，然后执行

1.3. 衡量计算机系统指标：

字长，内存容量，运算速度（MIPS（百万条指令/s）million I instruction per second）,时钟频率

简述计算机系统的组成	硬件系统 + 软件系统；硬件包括 CPU、存储器、I/O；软件包括系统软件和应用软件

2.微型计算机系统基础

2.1. 计算机为什么采用二进制？二进制有什么特点

（1） 二进制物理上容易实现：0 低电平，1 高电平

（2） 二进制便于编码计算

（3） 10 与逻辑中真假对应

2.2. 什么是指令，指令系统，程序

指令是对计算机的命令，指令系统包含所有的指令，程序则是指令的有序集合，可以执行程序达到目标。

2.3. 汇编语言，汇编程序，机器语言

机器语言是计算机可以直接读懂的指令/程序；汇编是用约定的文字、符号数字来表示不同的指令

2.4. 应用题：补码，8421bcd 码，二进制十进制八进制十六进制；地址首尾计算（注意末尾-1）。ASCII->CR（13），换行（10），空格（32），其余用‘0’,‘a’,‘A’表示即可。

ASCII：

CR：13；LD：10；SPACE：32；0:30H；A：41H；a：61H

3.8086

3.1. 地址

（1） 字：16 位，字节 8 位；寻址能力：1MB

8086 约定字读取从偶地址开始读为规则字。

读取方案：

字节：～BHE=0，A0=1 读奇地址，反之偶地址

字：

若读规则字：～BHE=0，A0=0

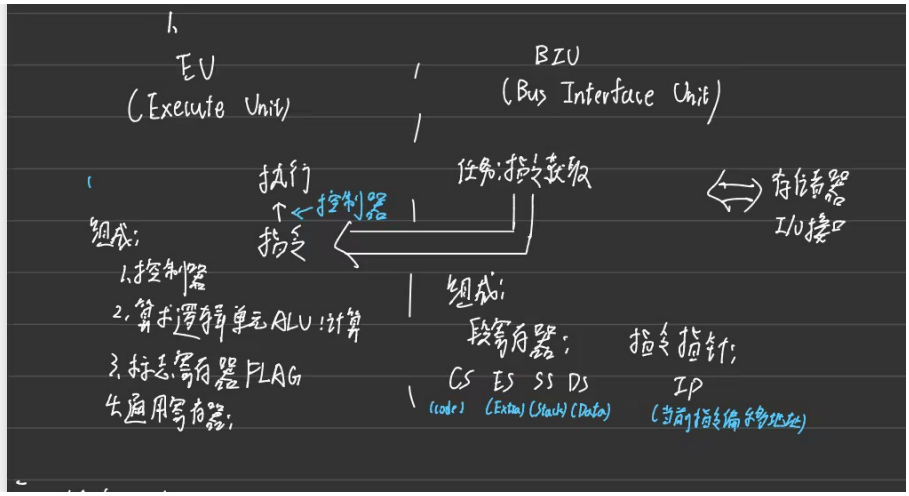
不规则字：本质上是读俩次字节，先读奇地址再偶。

（2） 存储器分段存储：

段地址*10H+偏移地址，实现读 20 位物理地址（真实存储器的地址）

逻辑地址 段地址：偏移地址，物理地址：真实物理空间地址

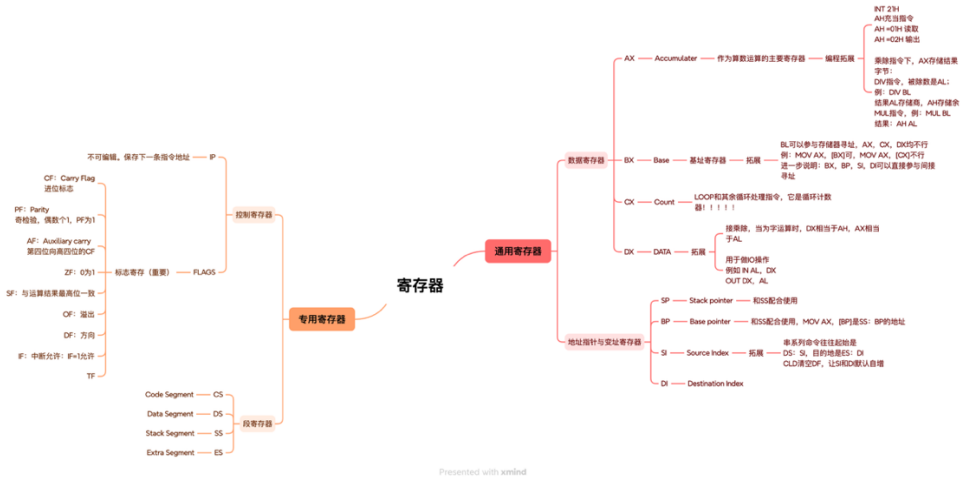
3.2. 组成



BIU：负责存储器与 IO 口信息读写，获取指令并排队

EU：从指令队列取出指令，执行。

3.3. 寄存器结构



FLAGS：控制标志：DF，TF，IF，其他为状态标志

程序中，状态标志位反应当前运算和操作结果的状态调节。而控制标志位用来控制 cpu 操作

3.4. 工作周期

T1 传输地址，T2T3T4 传输数据，Tw 等待，Tw 插入在 t3，t4 中

问：8086 为什么采用数据线和地址线复用，因为时序规定号了，因此可以复用

3.5. 中断

（1）可屏蔽 INTR：IF=0 则屏蔽

不可屏蔽 NMI

优先级：

1. 软件中断除了单步中断
2. NMI
3. INTR
4. 单步中断

（2）中断向量表，每个中断指令占 4B，256 个，总共 1KB 空间。依据中断类型码查找；

中断向量是中断服务程序的入口地址，由 CS 和 IP 组成，共 4 个字节。

8086 的中断向量表位于内存最低 1KB，即 00000H 到 003FFH。每个中断类型号对应一个中断向量，中断类型号为 n 时，其中断向量地址为 4n。

当 CPU 响应中断时，会根据中断类型号到中断向量表中取出对应的 CS 和 IP，然后转去执行相应的中断服务程序。

（3） 中断处理操作：

a) 发送中断请求信号

b) 检查 IF

c) 执行：

将 FLAGS 压入，清空 IF 和 TF。保护断点：压入 CS 和 IP；读取中断类型号，找到中断程序入口；执行程序；返回

3.6. 最小工作与最大工作状态

MN/～MX 切换最小最大工作状态

会使用：

最小工作状态：A0-A19

读：～RD，写：～WR，存储器读/IO：M/～IO

～BHE。

DT/～R：控制传输信号方向，1 发送

ALE：地址锁存信号：T1 时有效

～DEN：数据选通信号（允许传输）

最大工作模式：A0-A19，S0-S2 供 8288 翻译

IO 读写：～IORC，～IOWC

存储器读写：～MRDC，MWTC

区别：最小工作模式，8086 直接产生全部总线控制信号和命令输出信号，并提出访问总线的逻辑信号。最大工作模式：通过 8288 读区 8086 的状态信号，输出读写控制命令，实现最佳系统性能

3.7. IO 编址方式：单独编址

3.8. 8086 复位后：

IP，DS，SS，ES，指令队列全 0，CS 全 1.处理器从 FFFF0H 开始

3.9. 总线数据为什么要对部分地址线锁存，用什么信号锁存

由于 8086AD0-16 是地址数据分时复用的，因此需要 ALE 电平锁存信号，将在 T1 传输的地址线信息锁存。

3.10. 8086 与慢速外设交换数据时，为什么要插入等待周期 Tw？

8086 的总线周期一般由 T1、T2、T3、T4 组成。

当存储器或外设速度较慢，不能在规定时间内完成数据传送时，就需要通过 READY 信号通知 CPU 等待。

如果 READY 无效，CPU 会在 T3 和 T4 之间插入一个或多个等待周期 Tw。

插入 Tw 的作用是延长总线周期，使慢速外设有足够时间完成数据准备或接收，从而保证数据传送正确。

4.寻址方式与指令系统

4.1. 寻址方式!!!

立即寻址 MOV AX，251，相当于立即输入了结果。

寄存器寻址 MOV AX，BX 把 BX 的内容放进 AX

存储器寻址

直接寻址 MOV AL，Table+2 相当于把 table+2 的地址内容读取给 AL

寄存器间接寻址 MOV AX,[BX], 相当于 BX 保存了一个地址，接下来用这个地址去寻找那个地址中的内容给 AX

寄存器相对寻址 MOV AX,Table[SI],相当于 c 语言数组的下标,Tabley 已知，加一个寄存器保存的

基址加变址寻址 MOV AX，[BX+SI]，与上一个不一样，这个是俩个寄存器，没有已知。

相对基址加变址寻址，MOV AL，TABLE[BX][SI]，1+1 等于 2，但是名字不一样

注意上述的间接寻址，都是只能用 BX，BP，SI，DI

IO 寻址：

IN AL，DX

OUT DX，AL

4.2. 指令系统!!!

会用：

MOV，PUSH，POP（PUSH 让 SP 自减）

LEA 保存相对地址

XCHG DST，SRC

ADD，INC，SUB，DEC，CMP 基础

MUL 有符号数的乘法，IMUL 无符号数

ADC，SBB 带进位/借位的计算

调整指令：

DAA，DAS：压缩 BCD（字节存储的 BCD 码）加法减法后调整为 BCD

AAA，AAS：非压缩同理

AND，OR，XOR，TEST，NOT

SAL SAR：算数右移，不改变符号，除 2

SHR，SHL，逻辑左移/右移

ROL，ROR，小循环，出来再进去

上述 CF 都保存排出来的

RCL，RCR，大循环，相当于 CF 加进去，变成循环（注意循环量是 CL）

REP，REPE，REPNE 搭配使用，均循环 cx 次：

MOVSB，CMPSB，SCASB；MOVSW，CMPSW，SCASW

字节和字。DS：SI 起点，ES：DI 终点；AL 作为中间量

JMP：

JC：CF
JS：SF
JO：OF
JZ：ZF
JP：PF
无符号比较：
JA：ABOVE
JB：below
有符号：
JG：greater
JL：lower

LOOP：
LOOP
LOOPE

CALL：
调用您的函数。
RET ：回来吧～

标志位清理：
CLC 清 cf，CLD 清 df，NOP 延时一下
STC CF1，STD DF1

MOV DX，字符串偏移，$结尾
AH，09H
段内间接寻址：
JMP WORD PTR 存储器寻址方式
JMP FAR PTR
JMP DWORD PTR

会读：
XLAT！ AL<-(BX+AL)，类似读取表格的第 AL 个
AAM，AAD

访问类型	默认段寄存器
取指令	CS:IP
堆栈操作	SS:SP
一般数据访问	DS

访问类型	默认段寄存器
BP 作基址	SS
串操作源串	DS:SI
串操作目的串	ES:DI

简述汇编语言的特点	面向机器；效率高；可直接控制硬件；程序可读性比机器语言好；但可移植性差、编程复杂
为什么汇编程序通常比高级语言程序效率高	汇编指令接近机器指令，可精确控制寄存器、内存和 I/O；减少不必要的中间代码和编译开销；但开发效率低
简述机器语言、汇编语言、高级语言区别	机器语言为二进制指令；汇编语言用助记符表示机器指令；高级语言更接近自然语言和算法描述
简述寻址方式的意义	指令中说明操作数所在位置或获得操作数的方法；决定 CPU 如何取操作数
简述立即寻址和直接寻址区别	立即寻址操作数直接写在指令中；直接寻址指令中给出内存偏移地址，操作数在内存中
简述寄存器寻址和寄存器间接寻址区别	寄存器寻址操作数在寄存器中；寄存器间接寻址时寄存器中存的是偏移地址，操作数在内存中
简述基址、变址、基址变址寻址	基址用 BX/BP；变址用 SI/DI；基址变址用基址寄存器加变址寄存器形成有效地址

5.汇编语言

用第四章的东西。

MOV 4CH
INT 21H 结束

MOV AH，09H 字符串输出
MOV AH，0AH 输入
DX 保存偏移量

6.半导体存储

6.1. 分类

存储介质：半导体：体积小功耗低，存取效率高。易失性
磁介质：非易失性
光盘：非易失性，可靠，可互换
存取方式：
RAM：随机存储器（random access Memory）
ROM：只读存储器，不能重新写入：存放固定不变差程序、常数、汉字字库

6.2. 分级存储结构

存储器分级结构是把不同速度、容量和价格的存储器按层次组成一个系统，一般包括寄存器、Cache、主存和外存。

6.3. 存储性能指标

存储器容量：存储器单元数*位数：1K*8=1KB
存取周期：完整实现读出或写入的时间
功耗
可靠性
集成度：描绘构成相同容量存储器的芯片数量

6.4. SRAM

考察：连线（位扩展和字扩展如何到达指定的存储空间）
注意事项：
单一 2114 1K*4 位
6264:8K*8 位
和 8086 连接：
A0～A19
D0～D8:数据线
～CS 选中芯片
最小工作模式：
M/～IO 选中存储器
～WR 写，～RD 读；与 2114 的～WE 连接，～WE 状态和～WR 一样，6264 以～OE 为读状态
最大工作模式：
S0—S2 接 8288，～MWTC 写，～MRDC 读

6.5. DRAM：动态随机存取存储器

与 SRAM 区别：DRAM 因为使用电容稳定存储信息，而电容会随时间电压流失，所以需要刷新存储信息，才能保存数据。刷新以行为单位操作。刷新同时读写则仲裁。

共同：均有易失性

DRAM：价格便宜，速度慢，容量大，用于做主存储器；SRAM 做 cache，速度快

6.6. ROM

分类：MROM（Masked ROM）：制成后无法更改

PROM（Programmed ROM）：可编程只读存储器，用户只能写一次

EPROM（Erasable Programmed ROM）：用紫外线照射清除整片数据，然后再写入

EEPROM（Electrically Erasable Programmed ROM）：可以以字节为单位擦写

Flash Memory：快版 EEPROM，但是只能分块/全片擦出

大批量生产，程序永远不变	MROM	成本最低，出厂写死
小批量产品，程序只需要写入一次	PROM	用户可一次性写入，之后不改
开发调试阶段，需要反复擦写，但不频繁	EPROM	可擦可写，但紫外线擦除麻烦
设备运行中需要少量参数经常修改	EEPROM	可电擦写，支持字节级修改
需要大容量、较快擦写，如 U 盘、BIOS、SSD	Flash	块擦除，容量大，速度快，成本低

简述存储器分级结构及优点	寄存器、Cache、主存、外存逐级构成；速度逐渐降低，容量逐渐增大，成本逐渐降低；兼顾速度、容量、成本
简述 RAM 和 ROM 的区别	RAM 可读写，一般易失；ROM 通常只读或少量改写，非易失；RAM 作主存工作区，ROM 存放固化程序
简述 SRAM 和 DRAM 的区别	SRAM 用触发器，速度快、无需刷新、成本高；DRAM 用电容，容量大、成本低、需刷新
简述存储器扩展的基本方法	位扩展增加数据位宽；字扩展增加存储单元数量；字位扩展同时增加容量和位宽

简述存储器地址译码的作用	根据高位地址线产生片选信号，使 CPU 访问指定存储芯片；避免地址冲突
简述全译码和部分译码区别	全译码使用全部相关高位地址线，地址唯一；部分译码只使用部分高位地址线，电路简单但可能产生地址重叠
简述 Cache 的作用	利用程序局部性，在 CPU 和主存之间加入高速缓存，提高访问速度，缓解 CPU 与主存速度不匹配

7.输入输出技术

7.1. 为何需要？

各自外围设备间差异明显，输入输出方式与具体设备无关。

7.2. 程序控制方法

基本方法：

程序实现：程序查询，中断控制

硬件实现：DMA，通道，外围机

无条件传送方式：简单外设作为输入输出设备，前提：外设时刻已经就绪，适合信号变化很缓慢的情况，传送频率低

程序查询方式：不断查询外设的状态，缺点：cpu 要被占用较多时间。优点：程序简单，硬件简单。

流程：

启动外设->读外设状态->若准备好->传送数据->完成

中断控制方式：外设准备好了主动向 cpu 发送信号，提高 cpu 利用率，使 cpu 外设同时工作，适合低速，随机事件外设

中断判别优先级方式：

软件查询：读取中断请求触发器状态，然后按照优先级条件判别

硬件判别：例如使用比较器，比较当前执行中断和下一条的优先级顺序

7.3. 外设接口通外设之间的信息

信息通过接口传输，传输时 cpu，外设之间的电路，协调外设和 cpu

数据信息：OUT 输出，IN 输入，包含的是数据

控制信息：控制外设

状态信息：外设的工作状态

7.4. 接口编码方式

I/O 端口有两种编址方式：统一编址和独立编址。

统一编址是把 I/O 端口和存储器放在同一个地址空间中，访问 I/O 端口可以使用访问存储器的指令，指令形式比较灵活。但它会占

用一部分存储器地址空间。

独立编址是 I/O 端口和存储器分别使用不同的地址空间，访问 I/O 端口要用专门的输入输出指令，如 IN、OUT。它不占用存储器地址空间，但指令种类相对较少。

8.常用接口芯片

8.1. 并行通信与串行通信

直观：并行就是多根线，一次传多位

串行就是一根线，分时序传递信号

并行，通信线多，成本高，适合近距离

串行适合网络传输数据

注意：在计算机内部都是并行通信传递

8.2. IO 接口电路应该包含什么

和普通接口芯片差别：多种工作方式，可编程

1. 数据端口
2. 控制寄存器，状态寄存器
3. 中断控制逻辑
4. 地址译码器
5. 读写控制逻辑

8.3. 8255！

结构：～CS 选中，～RD 读，～WR 写，A0，1；DB 数据线

内部结构：A 组 8 个端口

B 组 8 个

C 组上 4，下 4

具体实现：

A1，A0=00 选中 A，01 选中 B，10 选中 C，11 写控制寄存器

控制方式 0：

控制字：

D7	D6	D5	D4	D3	D2	D1	D0
1	端口 A 方式		端口 A	端口 C 高位	端口 B 方式	端口 B	端口 C 低

00:方式 0..　　1 输入，0 输出

D7	D6 -D4	D3 -D1	D0
0	X	选择 PC 端口	1:Set。0:Reset

三个方式工作差别：

方式 0：基本输入输出

方式 1:能握手，握手输入输出
方式 2:cpu 能传输数据给外设，外设也能给 cpu

8255A 是一种可编程并行 I/O 接口芯片，用于 CPU 和外设之间的并行数据传送。

8255A 有三个 8 位并行端口：A 口、B 口和 C 口。其中 C 口还可以分成高 4 位和低 4 位分别使用。

8255A 有三种基本工作方式：方式 0、方式 1 和方式 2。

方式 0 是基本输入输出方式，端口可以直接作为输入口或输出口使用，不需要联络信号，常用于简单外设的数据传送。

方式 1 是选通输入输出方式，需要利用 C 口的部分引脚作为联络信号，实现带握手信号的数据传送。

方式 2 是双向传送方式，主要用于 A 口，可以实现双向数据传送。

简述 8255A 方式 0 的特点。

8255A 的方式 0 是基本输入输出方式。

在方式 0 下，A 口、B 口、C 口可以通过控制字分别设置为输入或输出。

方式 0 不需要握手联络信号，数据直接输入或输出。

它适合连接简单外设，如开关、LED、数码管等。

方式 0 的特点是结构简单、使用方便，但不能自动完成复杂的联络控制。

1. 简述 8086CPU 中 BIU 和 EU 的主要作用。

8086CPU 从功能上分为 BIU 和 EU 两部分。

BIU 是总线接口单元，主要负责形成 20 位物理地址，完成取指令、读写存储器和 I/O 端口等总线操作，并把取来的指令放入指令队列。

EU 是执行单元，主要负责从指令队列中取出指令，进行译码和执行，完成算术逻辑运算，并根据运算结果修改标志寄存器。

BIU 和 EU 可以并行工作，从而提高 CPU 的执行效率。

---

2. 简述 8086 物理地址的形成方法，并说明采用段地址和偏移地址的原因。

8086 有 20 根地址线，可以寻址 1MB 存储空间，但内部寄存器是 16 位的，不能直接表示 20 位地址。

因此 8086 采用段地址和偏移地址形成物理地址。

公式为：

物理地址 = 段地址 \times 10H + 偏移地址

也就是段地址左移 4 位后，再加上偏移地址。

采用这种方式可以用两个 16 位地址形成 20 位物理地址，扩大寻址范围，也便于程序按段进行管理。

---

3. 简述 CF 和 OF 标志位的区别。

CF 是进位标志，主要用于无符号数运算。

加法最高位产生进位，或减法最高位产生借位时，CF 置 1，否则为 0。

OF 是溢出标志，主要用于有符号数运算。

两个同号数相加，结果符号与原来相反时，或者两个异号数相减结果符号异常时，OF 置 1。

所以，CF 反映无符号数是否超出范围，OF 反映有符号数是否溢出。

---

4. 简述 8086 的最小工作方式和最大工作方式的区别。

8086 有最小工作方式和最大工作方式，由 MN/MX 引脚决定。

最小工作方式用于单处理器系统，8086 自己直接产生主要的总线控制信号，如 RD、WR、M/IO 等，硬件结构比较简单。

最大工作方式用于多处理器或协处理器系统，8086 输出状态信号，由总线控制器 8288 产生存储器读写、I/O 读写等控制信号。

所以，最小方式适合单 CPU 系统，最大方式适合较复杂的多处理器系统。

5. 简述 8255A 方式 0 的特点。

8255A 的方式 0 是基本输入输出方式。

在方式 0 下，A 口、B 口和 C 口可以通过控制字分别设置为输入或输出。

方式 0 不需要握手联络信号，数据可以直接输入或输出。

它适合连接简单外设，如开关、LED、数码管等。

方式 0 的特点是结构简单、使用方便，但不能自动完成复杂的联络控制。

1. 请描述 8086 微处理器的寄存器结构，并解释其主要寄存器的作用。

8086 的寄存器主要包括通用寄存器、段寄存器、指针和变址寄存器、指令指针寄存器和标志寄存器。

通用寄存器有 AX、BX、CX、DX，主要用于存放数据和参与运算。其中 AX 常作累加器，BX 常作基址寄存器，CX 常作计数器，DX 常用于 I/O 端口地址和乘除法运算。

段寄存器有 CS、DS、SS、ES，分别表示代码段、数据段、堆栈段和附加段的段地址。

指针和变址寄存器有 SP、BP、SI、DI，SP 指向栈顶，BP 常用于访问堆栈，SI 和 DI 常用于变址寻址和串操作。

IP 为指令指针寄存器，存放下一条指令的偏移地址。FLAGS 为标志寄存器，用于反映运算结果状态和控制 CPU 工作。

---

2. 请描述汇编语言的特点，并解释为什么汇编程序比高级语言程序运行效率更高。

汇编语言是一种面向机器的低级语言，与 CPU 指令关系密切，可以直接操作寄存器、存储器和 I/O 端口。

它的特点是执行速度快、占用存储空间少、能直接控制硬件，但可读性和可移植性较差，编程比较复杂。

汇编程序效率高，是因为汇编指令和机器指令基本一一对应，程序员可以直接控制硬件资源，减少了高级语言编译后产生的额外指令和运行开销，所以运行速度通常更快。

---

3. 请解释什么是中断向量，并描述其在 8086 微处理器中的作用。

中断向量是中断服务程序的入口地址，由 CS 和 IP 组成，共 4 个字节。

8086 的中断向量表位于内存最低 1KB，即 00000H 到 003FFH。每个中断类型号对应一个中断向量，中断类型号为 n 时，其中断向量地址为 4n。

当 CPU 响应中断时，会根据中断类型号到中断向量表中取出对应的 CS 和 IP，然后转去执行相应的中断服务程序。

---

4. 简述 8086 汇编语言中立即寻址方式和直接寻址方式的区别，并各举一例说明。

立即寻址方式是指操作数直接写在指令中，操作数本身就是一个常数。

5. 简述存储器分级结构的实现原理及其优势。

存储器分级结构是把不同速度、容量和价格的存储器按层次组成一个系统，一般包括寄存器、Cache、主存和外存。

它的实现原理主要是利用程序的局部性原理。CPU 先访问速度快、容量小的高层存储器，如果没有找到需要的数据，再到低层存储器中寻找，并把常用数据调入高层存储器。

这种结构的优点是可以提高平均访问速度，扩大存储容量，同时降低系统成本，使存储器系统兼顾速度、容量和价格。

1. 简述 8086 为什么采用地址/数据总线复用？ALE 信号有什么作用？

8086 为了减少芯片引脚数量，采用地址/数据总线复用方式。

其中 AD0～AD15 在总线周期的 T1 状态传送地址，在后面的状态传送数据。

由于地址和数据分时使用同一组引脚，所以需要把 T1 状态出现的地址信息锁存下来。

ALE 是地址锁存允许信号，在 T1 状态有效，用来控制地址锁存器锁存地址信息。

这样即使 AD 线后面转为数据线使用，外部仍然可以保持稳定的地址信号。

2. 简述 8086 中断响应后 CPU 自动完成的主要操作。

CPU 响应中断时，先执行完当前指令，然后保护断点和现场。

8086 会把 FLAGS、CS、IP 依次压入堆栈，并清除 IF 和 TF 标志。

然后根据中断类型号查中断向量表，取出对应中断服务程序的入口地址，即新的 CS 和 IP。

接着 CPU 转去执行中断服务程序。

中断服务程序结束时执行 IRET，从堆栈中恢复 IP、CS 和 FLAGS，返回原程序继续执行。

8.4. 3. 简述

8.5. CALL
8.6. 、
8.7. RET
与堆栈的关系。

CALL 是子程序调用指令，执行时 CPU 会把返回地址压入堆栈，然后转去执行子程序。

若是段内调用，一般压入返回地址 IP；若是段间调用，则压入 CS 和 IP。

RET 是子程序返回指令，执行时 CPU 从堆栈中弹出返回地址，使程序返回到调用点后继续执行。

因此堆栈用于保存子程序调用时的返回地址，保证程序能够正确返回。

8.8. 4. 简述

8.9. INT
8.10. 和
8.11. IRET
8.12. 指令的作用。

INT 是软件中断指令，用于产生一个指定类型号的中断。

执行 INT n 时，CPU 会保护现场，把 FLAGS、CS、IP 压入堆栈，然后根据中断类型号 n 查中断向量表，转去执行相应的中断服务程序。

IRET 是中断返回指令，用于从中断服务程序返回原程序。

执行 IRET 时，CPU 从堆栈中依次弹出 IP、CS、FLAGS，恢复中断前的断点和标志状态。

8.13. 5. 简述 8086 中

8.14. PUSH
8.15. 和
8.16. POP
8.17. 指令的执行过程。

8086 的堆栈由 SS 和 SP 指定，栈顶地址为 SS:SP。

执行 PUSH 指令时，SP 先减 2，然后把一个字数据压入 SS:SP 指向的栈顶单元。

执行 POP 指令时，CPU 先从 SS:SP 指向的栈顶单元取出一个字数据，然后 SP 加 2。

所以 8086 堆栈按字操作，并且栈顶向低地址方向增长。

8.1. 7. 简述存储器位扩展和字扩展的区别。

位扩展是为了增加存储器的数据位数，也就是扩展字长。

例如用两片 1K×4 的芯片并联，可以组成 1K×8 的存储器。两片芯片共用地址线和控制线，数据线分别接低 4 位和高 4 位。

字扩展是为了增加存储单元数量，也就是扩展存储容量。

例如用多组 1K×8 芯片，通过高位地址译码分别产生片选信号，可以组成更大容量的存储器。

所以，位扩展扩展数据位宽，字扩展扩展地址范围和存储容量。

---

8. 简述存储器地址译码的作用。全译码和部分译码有什么区别？

地址译码的作用是根据 CPU 发出的高位地址线产生片选信号，选中对应的存储器芯片或接口芯片。

这样可以使不同芯片对应不同的地址范围，避免多个芯片同时占用数据总线。

全译码是使用全部相关高位地址线参与译码，每个存储单元对应唯一地址，不会产生地址重叠。

部分译码是只使用部分高位地址线参与译码，电路较简单，但可能出现多个地址对应同一芯片的情况，即地址重叠。

---

9. 简述 SRAM 和 DRAM 的区别。

SRAM 是静态随机存储器，一般用触发器保存信息。

它的速度快，不需要刷新，但集成度较低，成本较高，常用于 Cache。

DRAM 是动态随机存储器，一般用电容保存信息。

由于电容上的电荷会逐渐泄漏，所以 DRAM 需要定期刷新。它速度相对较慢，但集成度高、容量大、成本低，常用于主存。

二者都属于 RAM，一般都是易失性存储器。

---

10. 简述 ROM 的主要类型及特点。

ROM 是只读存储器，通常用于存放固定程序、常数和系统启动程序，断电后信息不丢失。

MROM 是掩膜 ROM，内容在生产时写入，用户不能修改。

PROM 是可编程 ROM，用户可以写入一次，写入后不能再修改。

EPROM 可用紫外线擦除后重新写入，适合开发调试。

EEPROM 可用电信号擦写，通常可以按字节修改。

Flash 是一种可电擦写 ROM，容量大、速度较快，但一般按块擦除，常用于 U 盘、BIOS、SSD 等。

---

11. 简述程序查询方式和中断方式的区别。

程序查询方式是 CPU 不断查询外设状态，判断外设是否准备好。如果准备好，就进行数据传送。

这种方式程序简单、硬件要求低，但 CPU 需要反复等待和查询，效率较低。

中断方式是外设准备好后主动向 CPU 发出中断请求，CPU 响应中断后转去执行中断服务程序完成数据传送。

这种方式 CPU 不需要一直等待外设，可以提高 CPU 利用率，但硬件和程序结构相对复杂。

8. 简述存储器的主要性能指标。

存储器的主要性能指标包括容量、存取速度、功耗、可靠性和集成度。

容量表示存储器能够存放信息的多少，常用字节数表示。

存取速度通常用存取时间或存取周期表示，反映读写一次数据所需的时间。

功耗表示存储器工作时消耗的电能。

可靠性表示存储器长期正确保存和读写数据的能力。

集成度表示单位芯片中能够集成的存储单元数量，集成度越高，相同容量所需芯片数越少。

10. 简述为什么计算机采用二进制。

计算机采用二进制，主要是因为二进制在物理上容易实现。

二进制只有 0 和 1 两种状态，可以用低电平和高电平表示，电路实现简单、可靠。

二进制还便于进行逻辑运算，可以和逻辑中的假、真相对应。

另外，二进制运算规则简单，适合计算机内部进行编码、存储和运算。

---

11. 简述冯·诺依曼计算机的基本思想。

冯·诺依曼计算机的基本思想是存储程序控制。

也就是把程序和数据都用二进制形式表示，并存放在同一个存储器中。

计算机工作时，按地址顺序从存储器中取出指令并执行。

冯·诺依曼计算机由运算器、控制器、存储器、输入设备和输出设备组成。

早期冯·诺依曼计算机以运算器为中心，现代计算机一般以存储器为中心。

---

12. 简述指令、指令系统和程序的区别。

指令是 CPU 能够识别和执行的操作命令，用来规定计算机完成某种基本操作。

指令系统是某种 CPU 所能执行的全部指令的集合。

程序是为完成某一任务而按一定顺序编写的一组指令。

所以，指令是基本操作单位，指令系统是所有指令的集合，程序是指令的有序组合。

---

13. 简述机器语言、汇编语言和高级语言的区别。

机器语言是用二进制代码表示的语言，CPU 可以直接执行，但人很难阅读和编写。

汇编语言用助记符表示机器指令，比机器语言容易理解，但仍然依赖具体 CPU，移植性较差。

高级语言更接近自然语言和算法描述，便于编程和移植，但通常需要编译或解释后才能执行。

一般来说，机器语言最接近硬件，高级语言最接近人的思维，汇编语言介于二者之间。

---

14. 简述微型计算机系统的组成。

微型计算机系统由硬件系统和软件系统组成。

硬件系统主要包括 CPU、存储器、输入设备、输出设备和总线等。

其中 CPU 包括运算器和控制器，负责执行指令和控制系统工作。

软件系统包括系统软件和应用软件。

系统软件如操作系统、编译程序等，应用软件是为完成具体任务而编写的软件。

---

15. 简述 I/O 接口的作用。

I/O 接口是 CPU 和外设之间的连接电路。

由于外设和 CPU 在速度、信号格式、工作方式等方面存在差异，不能直接相连，因此需要 I/O 接口进行协调。

I/O 接口可以完成数据缓冲、信号转换、地址译码、读写控制和状态检测等功能。

通过 I/O 接口，CPU 可以向外设输出控制信息，也可以从外设读取状态信息和数据信息。

---

16. 简述 I/O 接口中数据信息、状态信息和控制信息的区别。

数据信息是 CPU 和外设之间真正要传送的数据，例如键盘输入的数据或打印机输出的数据。

状态信息反映外设当前的工作状态，例如是否准备好、是否忙、是否出错等。

控制信息是 CPU 送给外设的命令，用来控制外设工作，例如启动、停止、复位或设置工作方式。

因此，数据信息用于传送内容，状态信息用于判断外设情况，控制信息用于控制外设动作。

---

17. 简述并行通信和串行通信的区别。

并行通信是一次同时传送多位数据，通常需要多根数据线。

它传送速度快，但连线多、成本高，适合近距离通信。

串行通信是按位依次传送数据，通常使用较少的信号线。

它传送速度相对较慢，但连线少、成本低，适合远距离通信。

计算机内部常采用并行通信，远距离外设通信常采用串行通信。

---

18. 简述无条件传送方式、程序查询方式和中断方式的区别。

无条件传送方式不检测外设状态，CPU 直接进行数据传送，适合总是准备好的简单外设。

程序查询方式是 CPU 不断查询外设状态，外设准备好后再进行数据传送。它程序简单，但占用 CPU 时间较多。

中断方式是外设准备好后主动向 CPU 发出中断请求，CPU 响应后再进行数据传送。它可以提高 CPU 利用率，但硬件和程序较复杂。

三者相比，无条件传送最简单，程序查询更可靠，中断方式效率更高。

---

19. 简述汇编语言程序的基本结构。

汇编语言程序一般由数据段、代码段和堆栈段等部分组成。

数据段用于定义变量、缓冲区和常数。

代码段用于存放程序指令，是程序执行的主体部分。

堆栈段用于保存返回地址、寄存器内容和临时数据。

程序中通常要用 ASSUME 指令说明段寄存器与各段之间的关系，并在程序开始时初始化 DS 等段寄存器。